SENIOR ENGINEER INTERVIEW PREP

Gokulakonda Gautham

DOCUMENT 8 OF 10

React, Frontend & Performance

11 Questions | Reconciliation | Hooks Deep Dive | TanStack Query | Vite | INP | SSR vs CSR

React, Frontend & Performance

Q1: Virtual DOM — what it is, how reconciliation works, and why React uses it.

The Virtual DOM (VDOM) is an in-memory JavaScript object tree that mirrors the real DOM. React uses it to batch and minimize actual DOM mutations, which are expensive.

Why DOM mutations are expensive: The real DOM is a C++ object tree exposed to JavaScript. Touching it triggers reflows (layout recalculations) and repaints (pixel rendering). These are synchronous and block the main thread. Minimizing DOM operations is critical for smooth 60fps UIs.

Reconciliation algorithm (Fiber): On every render, React creates a new VDOM tree. The reconciler (Fiber architecture since React 16) diffs the new tree against the previous one. It applies the minimum set of changes to the real DOM. The diffing is O(n) using two heuristics: elements of different types produce different trees (replace, don't patch), and keys identify which list items changed.

```
// React's reconciliation heuristic in action

// Without keys — React can't tell which item moved
// Every list item re-renders on reorder
<ul>
  {items.map(item => <li>{item.name}</li>)}
</ul>

// With stable keys — React knows exactly which items changed
<ul>
  {items.map(item => <li key={item.id}>{item.name}</li>)}
</ul>

// Key pitfall: never use array index as key for reorderable lists
// Index keys cause stale state — React reuses the component instance
// for the same index even when the underlying item has changed
{items.map((item, i) => <li key={i}>{item}</li>)}  // WRONG for dynamic lists
{items.map(item => <li key={item.id}>{item}</li>)}  // CORRECT
```

Fiber architecture: Fiber replaced the old synchronous stack reconciler. Fiber reconciliation is interruptible — React can pause reconciliation mid-tree to handle high-priority updates (user input) and resume later. This enables concurrent features like Suspense, useTransition, and useDeferredValue.

When VDOM diffing costs more than it saves: For very simple UIs or high-frequency updates (canvas animations, WebGL), the VDOM overhead is a net negative. Use direct DOM manipulation or libraries like SolidJS (no VDOM, compiles to fine-grained DOM updates) for these cases.

Q2: React rendering — when does a component re-render and how do you control it?

Understanding exactly what triggers a re-render is the foundation of React performance optimization. Most performance bugs come from components re-rendering when they don't need to.

Re-render triggers: 1. Own state changes (useState, useReducer). 2. Parent re-renders (even if props haven't changed — this is the default). 3. Context value changes. 4. Custom hook state changes. Notably: props changing alone doesn't trigger re-render — the parent re-rendering does, which passes new props.

React.memo: Wraps a functional component. Skips re-render if props are shallowly equal to previous render. Only useful if the parent re-renders frequently and the child's rendering is expensive. The memo comparison itself has a cost — don't wrap every component.

```
// Without memo: re-renders every time Parent re-renders
function ExpensiveChild({ data, onAction }) {
  console.log('ExpensiveChild rendered');
```

```
  return <div>{/* expensive render */}</div>;
}

// With memo: skips render if data and onAction haven't changed
const ExpensiveChild = React.memo(function ExpensiveChild({ data, onAction }) {
  return <div>{/* expensive render */}</div>;
});

// But this breaks memo — new function reference every Parent render:
function Parent() {
  const handleAction = () => doSomething();  // new ref every render
  return <ExpensiveChild data={data} onAction={handleAction} />;
}

// Fix with useCallback:
function Parent() {
  const handleAction = useCallback(() => doSomething(), []);
  return <ExpensiveChild data={data} onAction={handleAction} />;
}
```

Render vs commit phase: Render phase: React calls your component function, builds the VDOM tree. Can be interrupted (Concurrent Mode). Commit phase: React applies changes to the real DOM. Synchronous, cannot be interrupted. useLayoutEffect runs synchronously after commit; useEffect runs asynchronously after paint.

StrictMode double-render: In development, StrictMode intentionally renders components twice to detect side effects in render. If your render logs fire twice, that's why. Production renders only once.

Q3: useEffect — the full mental model, dependency array, and cleanup.

useEffect is the most misunderstood hook. The correct mental model is not 'this runs after render' — it's 'this synchronizes my component with an external system'.

Correct mental model: useEffect is for synchronization. After every render where the dependencies changed, React runs your effect to sync external state (subscriptions, timers, DOM nodes, fetch requests) with the current props/state. It's not a lifecycle method replacement — it's a synchronization primitive.

```
// Full effect lifecycle
useEffect(() => {
  // 1. Setup: runs after render when deps change
  const subscription = subscribe(userId);

  // 2. Cleanup: runs before next effect AND on unmount
  return () => {
    subscription.unsubscribe();  // prevent memory leak
  };
}, [userId]);  // re-run when userId changes

// Dependency array semantics:
useEffect(() => { ... });          // runs after EVERY render (usually wrong)
useEffect(() => { ... }, []);      // runs once after mount (misused as
componentDidMount)
useEffect(() => { ... }, [a, b]); // runs when a or b changes

// Common mistake: missing dependencies
function Profile({ userId }) {
  useEffect(() => {
    fetchUser(userId);  // userId used but not in deps
  }, []);  // stale closure — always fetches first userId

  // Correct:
  useEffect(() => {
    fetchUser(userId);
```

```
  }, [userId]);  // re-fetches when userId changes
}
```

Race condition in effects: If userId changes twice quickly, two fetches fire. The second may resolve before the first. Solution: use an AbortController or an 'active' flag in the cleanup.

```
useEffect(() => {
  let active = true;
  fetchUser(userId).then(user => {
    if (active) setUser(user);  // ignore stale responses
  });
  return () => { active = false; };
}, [userId]);

// Better: AbortController
useEffect(() => {
  const controller = new AbortController();
  fetch(`/api/user/${userId}`, { signal: controller.signal })
    .then(r => r.json()).then(setUser)
    .catch(e => { if (e.name !== 'AbortError') setError(e); });
  return () => controller.abort();
}, [userId]);
```

useLayoutEffect: Same API as useEffect but runs synchronously after DOM mutations, before the browser paints. Use for: reading DOM measurements (getBoundingClientRect), synchronously adjusting layout to prevent visual flicker. Misusing it causes janky UIs — default to useEffect.

Q4: useMemo vs useCallback — what they actually do and when to use them.

Both are memoization hooks, but they memoize different things. The key insight: they are performance optimizations, not correctness tools. Code should work without them — they just prevent unnecessary work.

useCallback: Memoizes a function reference. Returns the same function object across renders as long as dependencies haven't changed. Use when: passing a callback to a memoized child (React.memo), or as a dependency of another hook (useEffect, useMemo).

useMemo: Memoizes a computed value. Only recomputes when dependencies change. Use when: a computation is genuinely expensive (sorting/filtering large arrays, complex transformations), or to maintain referential equality for objects/arrays passed to memoized children.

```
// useCallback — stable function reference
const handleSubmit = useCallback(async (data) => {
  await api.post('/submit', data);
  onSuccess();
}, [onSuccess]);  // new function only if onSuccess changes

// useMemo — expensive computation
const sortedAndFiltered = useMemo(() => {
  return items
    .filter(item => item.category === selectedCategory)
    .sort((a, b) => a.price - b.price);
}, [items, selectedCategory]);  // recomputes only when these change

// useMemo for referential equality
// Without memo: new object every render breaks React.memo on child
const config = { theme: 'dark', lang: 'en' };  // new ref every render

// With memo: same object reference if values unchanged
const config = useMemo(() => ({ theme, lang }), [theme, lang]);

// When NOT to use them:
// Don't wrap every function in useCallback — the overhead isn't worth it
// for components that aren't memoized children
```

```
// Profile first, optimize second
```

The memo trap: Overusing useMemo/useCallback is a common mistake. Each creates a closure, stores the previous deps array, and does a shallow comparison on every render. For cheap computations or non-memoized children, the overhead exceeds the benefit. The React team's guidance: write clean code first, profile with React DevTools Profiler, then add memoization where needed.

Q5: Context API vs Zustand vs Redux — state management tradeoffs.

State management is one of the most debated topics in React. The right answer depends on the scale of shared state, update frequency, and team size.

Context API: Built-in. Zero dependencies. Every consumer re-renders when the context value changes — even if the specific piece of state they use hasn't changed. Works well for low-frequency updates (theme, locale, auth user). Bad for high-frequency updates (mouse position, form state, list filters) — causes too many re-renders.

Zustand: Minimal, hook-based global store. Subscriptions are selector-based — components only re-render when their selected slice changes. No Provider wrapper needed. Tiny bundle (~1KB). Works outside React (useful for syncing with non-React code). Your resume lists Zustand — know it deeply.

```
// Zustand store
import { create } from 'zustand'

const useUserStore = create((set, get) => ({
  user: null,
  isLoading: false,
  error: null,

  fetchUser: async (id) => {
    set({ isLoading: true, error: null });
    try {
      const user = await api.getUser(id);
      set({ user, isLoading: false });
    } catch (e) {
      set({ error: e.message, isLoading: false });
    }
  },

  clearUser: () => set({ user: null }),
}))

// Selector — component only re-renders when user.name changes
const userName = useUserStore(state => state.user?.name)

// Multiple selectors
const { fetchUser, isLoading } = useUserStore(
  useShallow(state => ({ fetchUser: state.fetchUser, isLoading: state.isLoading }))
)
```

Redux Toolkit: The modern Redux (not the old verbose Redux). Opinionated: createSlice reduces boilerplate, RTK Query handles data fetching/caching. Best for: large teams with complex state, when you need time-travel debugging, or when RTK Query's caching logic replaces React Query.

Decision guide: Local state: useState. Shared low-frequency state: Context. Shared high-frequency or complex state: Zustand. Server state (fetching, caching, sync): TanStack Query. Large team with complex interactions: Redux Toolkit.

Q6: TanStack Query (React Query) — why it exists and how it works internally.

TanStack Query is not a state manager — it's a server state synchronization library. It solves a different problem: keeping your UI in sync with server data, with caching, background refresh, and error handling built-in.

The problem it solves: Without it, every component that needs remote data writes its own useEffect + useState + error handling + loading state + cache invalidation logic. This is duplicated, error-prone, and doesn't share cache between components. TanStack Query centralizes all of this.

Query key: The cache key for a query. Must be serializable. Should be hierarchically structured. ['users'] invalidates all user queries. ['users', 42] is the cache for user 42. ['users', 42, 'orders'] is their orders. Invalidating a parent key cascades to children.

```
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query'

// useQuery: fetch and cache server data
function UserProfile({ userId }) {
  const { data: user, isLoading, error, isFetching } = useQuery({
    queryKey: ['users', userId],
    queryFn: () => api.getUser(userId),
    staleTime: 5 * 60 * 1000,   // consider fresh for 5 minutes
    gcTime: 10 * 60 * 1000,     // keep in cache 10 minutes after unmount
    retry: 2,
    refetchOnWindowFocus: true,  // re-fetch when user returns to tab
  })
  // isFetching: background refresh happening (data still shown)
  // isLoading: no data yet (first fetch)
  if (isLoading) return <Skeleton />
  if (error) return <Error message={error.message} />
  return <div>{user.name}</div>
}

// useMutation: modify server data + invalidate cache
function UpdateUser() {
  const queryClient = useQueryClient()
  const mutation = useMutation({
    mutationFn: (data) => api.updateUser(data),
    onSuccess: (data, variables) => {
      // Invalidate — triggers background re-fetch
      queryClient.invalidateQueries({ queryKey: ['users', variables.id] })
      // Or optimistically update cache directly:
      queryClient.setQueryData(['users', variables.id], data)
    },
    onError: (error) => toast.error(error.message),
  })
  return <button onClick={() => mutation.mutate(formData)}>Save</button>
}
```

staleTime vs gcTime: staleTime: how long data is considered fresh. No background re-fetch during this window. gcTime (formerly cacheTime): how long unused cache entries are kept in memory after all subscribers unmount. Setting staleTime=Infinity with manual invalidation is a common pattern for data that changes only on explicit mutation.

Prefetching: queryClient.prefetchQuery() warms the cache before the user navigates to a page. Route-level prefetching in TanStack Router + TanStack Query: hover over a link → prefetch the destination's data → instant navigation.

Q7: SSR vs CSR vs SSG vs ISR — when to use each.

Rendering strategy directly impacts SEO, time-to-first-byte, bundle size, and infrastructure complexity. Understanding all four and their tradeoffs is expected at senior level.

CSR (Client-Side Rendering): Browser downloads JS bundle, React renders in the browser. Empty HTML until JS loads and runs. Bad for SEO (crawlers see empty HTML). Bad TTFB (slow initial render). Good for:

authenticated dashboards, admin panels, anything behind login where SEO doesn't matter. Your Chrome extension work is effectively CSR.

SSR (Server-Side Rendering): Server renders HTML per request, sends fully populated HTML. Fast first paint, good SEO. Server runs on every request — compute cost scales with traffic. Good for: dynamic content that varies per user/request (personalized feeds, search results).

SSG (Static Site Generation): HTML generated at build time. Served as static files from CDN. Fastest possible TTFB. Zero server compute at runtime. Bad for: frequently changing data (requires rebuild to update). Good for: marketing pages, docs, blogs, anything that doesn't change between deployments.

ISR (Incremental Static Regeneration): Next.js feature. Static pages that regenerate in the background after a configurable interval. Serves stale page immediately, regenerates for next request. Best of SSG and SSR for content that changes occasionally (product pages, news articles).

```
// Next.js rendering modes

// SSG — built at build time
export async function generateStaticParams() {
  const products = await getProducts()
  return products.map(p => ({ id: p.id }))
}
export default async function ProductPage({ params }) {
  const product = await getProduct(params.id)  // runs at build time
  return <Product data={product} />
}

// SSR — runs on every request
export const dynamic = 'force-dynamic'
export default async function DashboardPage() {
  const user = await getCurrentUser()  // per-request server fetch
  return <Dashboard user={user} />
}

// ISR — regenerate every 60 seconds
export const revalidate = 60
export default async function NewsPage() {
  const articles = await getLatestArticles()
  return <NewsFeed articles={articles} />
}
```

Hydration: SSR sends HTML; then the client loads React and 'hydrates' — attaches event handlers to the existing HTML. During hydration, the server and client renders must match exactly or React throws a hydration mismatch error. A common source of bugs: rendering Date.now() or window.innerWidth without SSR guard.

Q8: Why is Vite faster than CRA? How do bundlers and tree shaking work?

Your resume mentions migrating from CRA to Vite with an 80% Lighthouse improvement. Expect to explain exactly why Vite is faster — not just 'it uses ESM'.

CRA's problem: CRA uses Webpack, which bundles ALL modules into one (or a few) large JS files before serving. On startup, Webpack must resolve, transform, and bundle every module in the app. A medium app: 5-30 second dev server startup, 1-5 second HMR updates.

Vite's approach (dev mode): No bundling in development. Vite serves files as native ES modules directly to the browser. The browser itself resolves imports. Vite only transforms what the browser actually requests (on-demand). Startup: near-instant regardless of app size. HMR: updates only the changed module — sub-100ms updates.

Vite production build: Vite uses Rollup (not ESBuild) for production builds. Rollup produces optimized, tree-shaken bundles. ESBuild is used for transforms (TypeScript, JSX) — it's a Go-based transpiler, 10-100x faster than Babel.

Tree shaking: Dead code elimination. Bundlers statically analyze ES module imports/exports and remove unused exports from the final bundle. Requires ES modules (import/export) — CommonJS (require) cannot be tree-shaken because imports are dynamic. Named exports are tree-shakeable; default exports are less so.

```
// Tree shaking works with named exports
// utils.js
export function add(a, b) { return a + b }
export function multiply(a, b) { return a * b }  // never imported
export function divide(a, b) { return a / b }    // never imported

// main.js
import { add } from './utils'  // only 'add' included in bundle
// multiply and divide are tree-shaken out

// Lodash — the classic tree-shaking failure
import _ from 'lodash'              // imports ENTIRE lodash (~70KB)
import { debounce } from 'lodash'   // still imports all (CommonJS)
import debounce from 'lodash/debounce'  // correct — only debounce
// Or use lodash-es: import { debounce } from 'lodash-es'  // tree-shakeable

// Code splitting — lazy load routes
const Dashboard = lazy(() => import('./Dashboard'))
// Dashboard.js and its deps are a separate chunk
// Downloaded only when the user navigates to that route
```

Bundle analysis: Use rollup-plugin-visualizer or webpack-bundle-analyzer to see what's in your bundle. Common culprits: moment.js (330KB, replace with date-fns), entire icon libraries (import only what you use), duplicate dependencies from different package versions.

Q9: React performance optimization — profiling, lazy loading, debouncing.

Performance optimization should always be measurement-first. The React DevTools Profiler tells you exactly which components render, how long they take, and why. Optimize based on data, not intuition.

React DevTools Profiler: Records render timings for every component. Flame graph shows render duration. 'Why did this render?' shows the trigger. Ranked chart shows the most expensive renders. Always profile in production mode — development adds significant overhead.

Code splitting and lazy loading: Split your bundle at route boundaries. Each route is a separate chunk downloaded on demand. React.lazy() + Suspense handles the loading state. This is what transformed the Lighthouse score in your Apxor migration — the initial bundle was huge from CRA's lack of automatic code splitting.

```
// Route-level code splitting
import { lazy, Suspense } from 'react'
import { Routes, Route } from 'react-router-dom'

const Dashboard = lazy(() => import('./pages/Dashboard'))
const Settings  = lazy(() => import('./pages/Settings'))
const Reports   = lazy(() => import('./pages/Reports'))

function App() {
  return (
    <Suspense fallback={<PageSkeleton />}>
      <Routes>
        <Route path='/dashboard' element={<Dashboard />} />
        <Route path='/settings'  element={<Settings />} />
        <Route path='/reports'   element={<Reports />} />
      </Routes>
    </Suspense>
  )
}
```

```
// Debounce search input — avoid query on every keystroke
function SearchBar({ onSearch }) {
  const [query, setQuery] = useState('')

  const debouncedSearch = useMemo(
    () => debounce((q) => onSearch(q), 300),
    [onSearch]
  )

  const handleChange = (e) => {
    setQuery(e.target.value)
    debouncedSearch(e.target.value)  // fires 300ms after last keystroke
  }

  useEffect(() => () => debouncedSearch.cancel(), [debouncedSearch])
  return <input value={query} onChange={handleChange} />
}
```

Virtualization: Rendering 10,000 list items creates 10,000 DOM nodes — crushes performance. Virtualization renders only the visible rows (typically 20-50). Libraries: TanStack Virtual (headless, works with any UI), react-window, react-virtuoso. Essential for tables, feeds, and long lists.

Image optimization: Use Next.js Image component (automatic WebP conversion, lazy loading, correct sizing). For plain React: loading='lazy' attribute, correct srcset for responsive images. Images were 40%+ of the INP improvement in your CRA→Vite migration.

useTransition and useDeferredValue: React 18 concurrent features. useTransition marks a state update as non-urgent — React can interrupt it for user input. useDeferredValue creates a deferred version of a value — UI stays responsive while expensive renders catch up. Use for filtering large lists, search results, any heavy re-render triggered by user input.

Q10: INP, Core Web Vitals — what they measure and how to improve them.

Your resume specifically mentions reducing INP from ~500ms to negligible. Interviewers will probe this. You need to explain what INP is and precisely what you did to fix it.

Core Web Vitals: LCP (Largest Contentful Paint): how fast the main content loads. Target: < 2.5s. FID → INP (Interaction to Next Paint): how fast the page responds to user interaction. Target: < 200ms. CLS (Cumulative Layout Shift): how much the layout jumps around. Target: < 0.1.

INP specifically: INP measures the longest interaction delay across a user's entire session — click, tap, or keyboard input. It replaced FID (which only measured the first interaction). A high INP means the main thread was busy when the user interacted — JS was executing, blocking input handling.

What causes high INP: Large JS bundles that block parsing. Heavy synchronous component renders. Long useEffect chains on interaction. Third-party scripts (analytics, chat widgets) blocking the main thread. Unthrottled scroll/resize handlers.

```
// INP optimization techniques

// 1. Break up long tasks — yield to browser between chunks
async function processLargeList(items) {
  for (let i = 0; i < items.length; i++) {
    processItem(items[i])
    if (i % 100 === 0) {
      // yield to browser — allows input handling
      await new Promise(resolve => setTimeout(resolve, 0))
    }
  }
}

// 2. useTransition — mark expensive update as non-urgent
```

```
const [isPending, startTransition] = useTransition()
const handleFilter = (value) => {
  setFilterInput(value)           // urgent — updates input immediately
  startTransition(() => {
    setFilteredResults(           // non-urgent — can be interrupted
      items.filter(i => i.name.includes(value))
    )
  })
}

// 3. Lazy load heavy components
const HeavyChart = lazy(() => import('./HeavyChart'))

// 4. web workers for CPU-heavy tasks
const worker = new Worker('./compute.worker.js')
worker.postMessage({ data: largeDataset })
worker.onmessage = (e) => setResult(e.data)  // result without blocking UI
```

Measuring: Chrome DevTools Performance tab shows long tasks (>50ms) highlighted in red. Lighthouse gives INP score with diagnostics. Real-user monitoring: web-vitals npm package reports INP from actual users, send to your analytics.

Q11: Controlled vs uncontrolled components — and when to use each.

This distinction affects form performance, integration with third-party libraries, and the predictability of your UI state.

Controlled: React state is the single source of truth. Every keystroke calls onChange which calls setState which triggers re-render. The input value is always in sync with React state. You have full control: can validate on change, transform input, enforce format. Downside: re-render on every keystroke — can be expensive for large forms.

Uncontrolled: The DOM is the source of truth. Use a ref to read the value on submit, not on every change. No re-render on keystroke. Works well for simple forms, file inputs (always uncontrolled), and integrating with non-React DOM libraries.

```
// Controlled — state-driven
function ControlledForm() {
  const [email, setEmail] = useState('')
  const [error, setError] = useState(null)

  const handleChange = (e) => {
    const val = e.target.value
    setEmail(val)
    setError(val.includes('@') ? null : 'Invalid email')  // live validation
  }
  return (
    <input value={email} onChange={handleChange} />
    // value prop = controlled (React owns the value)
  )
}

// Uncontrolled — ref-driven
function UncontrolledForm() {
  const emailRef = useRef(null)

  const handleSubmit = (e) => {
    e.preventDefault()
    const email = emailRef.current.value  // read on submit only
    submitForm({ email })
  }
  return (
```

```
    <form onSubmit={handleSubmit}>
      <input ref={emailRef} defaultValue='' />
      // defaultValue (not value) = uncontrolled
    </form>
  )
}

// React Hook Form — best of both worlds
// Uncontrolled internally (no re-render per keystroke)
// Controlled API (validation, watch, field arrays)
const { register, handleSubmit, formState: { errors } } = useForm()
```
